

Phase 9 — Composability Test

Entropy regularisation vs. Batch-Norm, Dropout, Weight-Decay

thesis-IV working notes (HyMeKo)

2026-04-26

1 Purpose

Phases 6–8 demonstrated that `scalar_entropy_normalized` (Path B) and `entropy_target` ($H^* = 0.5$, Path A) universally Pareto-dominate the original `scalar_entropy` regulariser across nine datasets and five architectures. **Open question for the manuscript:** is the new regulariser *competitive* with mainstream techniques alone, and *complementary* when stacked on top of them?

Phase 9 answers that by running paired comparisons `baseline` vs `scalar_entropy_normalized` under each of five model + optimiser settings:

ID	Variant	Modification
V1	<code>vanilla</code>	base model, Adam(lr= 10^{-3}).
V2	<code>dropout</code>	+ <code>nn.Dropout(p=0.5)</code> after each ReLU
V3	<code>batch_norm</code>	+ <code>nn.BatchNorm1d</code> after each Linear
V4	<code>weight_decay</code>	vanilla model, Adam(wd= 10^{-4})
V5	<code>full_stack</code>	BN + Dropout + Adam(wd= 10^{-4})

The pair `baseline` vs `scalar_entropy_normalized` within each variant tells us whether entropy adds value *on top of* the existing regularisation. Cross-variant comparisons of mean accuracies tell us whether entropy alone is competitive.

2 Configuration matrix

Five variants $\times$ four datasets = **20 paired comparisons** (40 individual training runs). Estimated wall-clock ~ 11 h. Auto-fires after Views PH8 suite finished marker.

Dataset/arch	Original sign	Seeds	Epochs
<code>spirals</code>	strongly +	100	50
<code>circles</code>	sig. –	100	50
<code>mnist_small</code>	+ (anchor)	33	15
<code>mnist_capsnet</code>	sig. –	33	10

3 Per-cell bash commands

For each (dataset, variant, seeds, epochs) configuration the harness is invoked exactly as below. The base shell script is `python/benches/thesis_iv_hard/run_overnight_views_ph9.sh`.

3.1 Block A: synthetic 2-D (spirals + circles)

```
# V1 vanilla
python3 ../run_benchmark.py --datasets spirals \
    --arms baseline scalar_entropy_normalized \
```

```

--seeds 100 --epochs 50 --lam 0.1 --reg-every-n 10 --view dataflow

# V2 dropout
python3 ../run_benchmark.py --datasets spirals_drop \
--arms baseline scalar_entropy_normalized \
--seeds 100 --epochs 50 --lam 0.1 --reg-every-n 10 --view dataflow

# V3 batch norm
python3 ../run_benchmark.py --datasets spirals_bn \
--arms baseline scalar_entropy_normalized \
--seeds 100 --epochs 50 --lam 0.1 --reg-every-n 10 --view dataflow

# V4 weight decay
python3 ../run_benchmark.py --datasets spirals \
--arms baseline scalar_entropy_normalized \
--seeds 100 --epochs 50 --lam 0.1 --reg-every-n 10 --view dataflow \
--weight-decay 1e-4

# V5 full stack
python3 ../run_benchmark.py --datasets spirals_full \
--arms baseline scalar_entropy_normalized \
--seeds 100 --epochs 50 --lam 0.1 --reg-every-n 10 --view dataflow \
--weight-decay 1e-4

```

The same five commands repeat with `--datasets circles*` for the second synthetic block.

3.2 Block B: MNIST plain MLP

```

# V1, V2, V3
python3 ../run_benchmark.py --datasets {mnist_small | _drop | _bn} \
--arms baseline scalar_entropy_normalized \
--seeds 33 --epochs 15 --lam 0.1 --reg-every-n 10 --view dataflow

# V4 (vanilla + WD)
python3 ../run_benchmark.py --datasets mnist_small \
--arms baseline scalar_entropy_normalized \
--seeds 33 --epochs 15 --lam 0.1 --reg-every-n 10 --view dataflow \
--weight-decay 1e-4

# V5 full stack
python3 ../run_benchmark.py --datasets mnist_small_full \
--arms baseline scalar_entropy_normalized \
--seeds 33 --epochs 15 --lam 0.1 --reg-every-n 10 --view dataflow \
--weight-decay 1e-4

```

3.3 Block C: CapsMLP MNIST

```

# V1, V2, V3
python3 ../run_benchmark.py --datasets {mnist_capsnet | _drop | _bn} \
--arms baseline scalar_entropy_normalized \
--seeds 33 --epochs 10 --lam 0.1 --reg-every-n 10 --view dataflow

# V4 (vanilla CapsMLP + WD)
python3 ../run_benchmark.py --datasets mnist_capsnet \
--arms baseline scalar_entropy_normalized \
--seeds 33 --epochs 10 --lam 0.1 --reg-every-n 10 --view dataflow \
--weight-decay 1e-4

```

```
# V5 full stack
python3 ../run_benchmark.py --datasets mnist_capsnet_full \
  --arms baseline scalar_entropy_normalized \
  --seeds 33 --epochs 10 --lam 0.1 --reg-every-n 10 --view dataflow \
  --weight-decay 1e-4
```

4 Hypotheses for the paper claim

1. **Composability hypothesis (the killer finding).** The paired Δ from adding `scalar_entropy_normalized` on top of V2–V5 is *at least as positive* as on V1 (vanilla). If yes, the regulariser **stacks** with existing techniques without conflict, which is the core practical claim.
2. **Standalone competitiveness.** The absolute mean accuracy of V1+entropy is comparable to or better than the absolute mean accuracy of V2–V5 standalone (i.e. baseline arm under each variant). If yes, `scalar_entropy_normalized` on its own is a credible single-tool regulariser.
3. **Failure-mode mitigation.** On the negative datasets (`circles`, `mnist_capsnet`), none of dropout / BN / WD alone neutralises the failure mode of the original `scalar_entropy`. If only V2–V5 *with* entropy do, the regulariser is a unique tool, not a substitutable one.
4. **Variance reduction.** The variance-reduction signature should survive — and ideally compound — when entropy is added on top of dropout / BN / WD.

5 Acceptance criteria

For the paper to claim “composability with mainstream regularisers”, at least 3 of 4 dataset $\times$ variant cells in V5 (full stack) must show $\Delta \geq 0$ from adding `scalar_entropy_normalized`, and at least 2 of those must reach significance ($p < 0.05$).

For the paper to claim “failure-mode mitigation”, V1, V2, V3, V4 under `baseline` on `circles` and `mnist_capsnet` must all retain a directional negative effect; while V1–V5 under `scalar_entropy_normalized` must all be null or positive.

6 After phase 9

Refresh `RESULTS_VIEWS_SUITE.md` via the existing aggregator. The new rows will appear under their dataset name. Phase 9 cells are identifiable by the `*_drop`, `*_bn`, `*_full` dataset suffixes and (where applicable) by the non-zero `weight_decay` in the per-run config printout.